

Chapter 45 — Palindrome Patterns

0. What You Will Learn in This Chapter

Palindrome problems are very common in interviews because they combine many patterns:

- two pointers
- string cleanup
- recursion
- dynamic programming
- expand around center
- hashing
- greedy
- partitioning
- prefix/suffix tricks
- KMP
- trie preview
- rolling hash preview

A palindrome is a string that reads the same forward and backward.

Examples:

```
"madam"  
"racecar"  
"abba"  
"a"  
""
```

Non-examples:

```
"abc"  
"abca"  
"hello"
```

By the end of this chapter, you should understand:

- Basic palindrome checking
- Two-pointer palindrome validation
- Valid palindrome with filtering
- Deleting at most one character
- Expanding around center
- Counting palindromic substrings
- Longest palindromic substring
- Longest palindromic subsequence
- Minimum insertions to make palindrome
- Palindrome partitioning
- Minimum palindrome cuts
- Shortest palindrome using KMP
- Palindrome pairs concept
- How to decide between two pointers, center expansion, DP, KMP, and hashing

Core problems covered in this chapter:

1. Valid Palindrome
2. Valid Palindrome II
3. Palindrome Number

4. Reverse String / helper patterns
 5. Longest Palindromic Substring
 6. Palindromic Substrings
 7. Longest Palindromic Subsequence
 8. Minimum Insertion Steps to Make a String Palindrome
 9. Palindrome Partitioning
 10. Palindrome Partitioning II
 11. Shortest Palindrome
 12. Break a Palindrome
 13. Valid Palindrome III
 14. Count Different Palindromic Subsequences, preview
 15. Palindrome Pairs, preview
 16. Manacher's Algorithm, preview
-

1. What Is a Palindrome?

A palindrome is a sequence that reads the same from both sides.

Examples:

```
"aba"  
"abba"  
"racecar"  
"1221"
```

For a string `s` to be palindrome:

```
s[left] == s[right]  
s[left + 1] == s[right - 1]  
...
```

This naturally suggests two pointers.

2. Why Palindrome Problems Are Important

Palindrome problems test whether you can recognize symmetry.

They also test whether you know when to use:

Problem Type	Best Pattern
Basic check	two pointers
Ignore punctuation/case	two pointers with filtering
Delete at most one char	two pointers + helper
Count all palindromic substrings	expand around center
Longest palindromic substring	expand center / DP
Longest palindromic subsequence	DP

Problem Type	Best Pattern
Partition into palindromes	backtracking + palindrome table
Minimum cuts	DP
Shortest palindrome	KMP / rolling hash
Palindrome pairs	trie / hashing

3. Palindrome Substring vs Palindrome Subsequence

This is critical.

3.1 Palindromic Substring

A substring must be contiguous.

Example:

s = "bbbab"

Palindromic substrings include:

"bbb"

"bb"

"bab"

3.2 Palindromic Subsequence

A subsequence can skip characters.

Example:

s = "bbbab"

Longest palindromic subsequence is:

"bbbb"

Length:

4

This is not necessarily contiguous.

3.3 Pattern Difference

Task	Pattern
Longest palindromic substring	expand around center / DP
Longest palindromic subsequence	interval DP
Count palindromic substrings	expand around center
Count palindromic subsequences	DP, harder

4. When to Use Which Palindrome Technique

Clue	Technique
Check if entire string is palindrome	two pointers
Ignore non-alphanumeric	two pointers with skip
Delete at most one char	two pointers + helper
Find longest contiguous palindrome	expand around center
Count all contiguous palindromes	expand around center
Longest palindrome by deleting chars	subsequence DP
Minimum insertions to palindrome	longest palindromic subsequence
Generate all palindrome partitions	backtracking
Minimum palindrome cuts	DP + palindrome table
Add chars in front to make palindrome	KMP
Need many palindrome checks	precompute palindrome table
Pair words to form palindromes	trie/hash

5. Pattern Recognition Checklist

Ask:

17. Is the palindrome required to be contiguous?
 18. Is it a subsequence?
 19. Are we checking the entire string?
 20. Are we allowed to delete characters?
 21. Are we counting palindromes?
 22. Are we finding longest length/string?
 23. Do we need all partitions?
 24. Do we need minimum cuts?
 25. Are repeated palindrome checks needed?
 26. Do we need prefix/suffix palindrome?
 27. Does KMP or rolling hash help?
-

6. Core Template 1 — Basic Palindrome Check

```
bool isPalindrome(string& s, int left, int right) {
    while (left < right) {
        if (s[left] != s[right]) {
            return false;
        }

        left++;
        right--;
    }

    return true;
}
```

Used in:

- Valid Palindrome II
 - Palindrome Partitioning
 - Palindrome Pairs
 - many helper functions
-

7. Core Template 2 — Expand Around Center

Every palindrome has a center.

For odd-length palindrome:

center at one character

Example:

racecar
e

For even-length palindrome:

center between two characters

Example:

abba
bb

Template:

```
int expand(string& s, int left, int right) {
    int count = 0;

    while (left >= 0 && right < s.size() && s[left] == s[right]) {
        count++;
        left--;
        right++;
    }
}
```

```
    return count;
}
```

Call for each center:

```
expand(s, i, i);        // odd
expand(s, i, i + 1);    // even
```

8. Core Template 3 — Palindrome DP Table

```
vector<vector<bool>> buildPalindromeTable(string s) {
    int n = s.size();

    vector<vector<bool>> isPal(n, vector<bool>(n, false));

    for (int len = 1; len <= n; len++) {
        for (int l = 0; l + len - 1 < n; l++) {
            int r = l + len - 1;

            if (s[l] == s[r]) {
                if (len <= 2 || isPal[l + 1][r - 1]) {
                    isPal[l][r] = true;
                }
            }
        }
    }

    return isPal;
}
```

Used when you need many palindrome checks.

9. Core Template 4 — Longest Palindromic Subsequence

```
for (int len = 1; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        if (l == r) {
            dp[l][r] = 1;
        } else if (s[l] == s[r]) {
            dp[l][r] = 2 + dp[l + 1][r - 1];
        } else {
            dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
        }
    }
}
```

```
}  
}  
}
```

10. Full Problem Walkthrough 1 — LeetCode 125: Valid Palindrome

10.1 Problem Summary

Check whether a string is palindrome after:

- converting uppercase to lowercase
- removing non-alphanumeric characters

Example:

```
"A man, a plan, a canal: Panama"
```

Answer:

```
true
```

10.2 Pattern

Two pointers.

Skip invalid characters.

10.3 C++ Code

```
class Solution {  
public:  
    bool isPalindrome(string s) {  
        int left = 0;  
        int right = s.size() - 1;  
  
        while (left < right) {  
            while (left < right && !isalnum(s[left])) {  
                left++;  
            }  
  
            while (left < right && !isalnum(s[right])) {  
                right--;  
            }  
  
            if (tolower(s[left]) != tolower(s[right])) {  
                return false;  
            }  
  
            left++;  
            right--;  
        }  
  
        return true;  
    }  
};
```

```
}  
};
```

10.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

11. Full Problem Walkthrough 2 — LeetCode 680: Valid Palindrome II

11.1 Problem Summary

Return true if s can be palindrome after deleting at most one character.

Example:

s = "abca"

Delete b or c:

"aca" or "aba"

Answer:

true

11.2 Key Insight

Use two pointers.

At first mismatch:

skip left OR skip right

Only one deletion is allowed.

11.3 C++ Code

```
class Solution {  
public:  
    bool check(string& s, int left, int right) {  
        while (left < right) {  
            if (s[left] != s[right]) {  
                return false;  
            }  
  
            left++;  
            right--;  
        }  
  
        return true;  
    }  
};
```

```

bool validPalindrome(string s) {
    int left = 0;
    int right = s.size() - 1;

    while (left < right) {
        if (s[left] != s[right]) {
            return check(s, left + 1, right) ||
                   check(s, left, right - 1);
        }

        left++;
        right--;
    }

    return true;
}
};

```

11.4 Why This Is $O(n)$, Not $O(n^2)$

The helper check runs at most twice, and only after the first mismatch.
So total work is linear.

12. Full Problem Walkthrough 3 — LeetCode 9: Palindrome Number

12.1 Problem Summary

Given integer x, return true if x is palindrome.

Example:

```

121 -> true
-121 -> false
10 -> false

```

12.2 Approach 1 — Convert to String

```

class Solution {
public:
    bool isPalindrome(int x) {
        string s = to_string(x);

        int left = 0;
        int right = s.size() - 1;

        while (left < right) {
            if (s[left] != s[right]) {
                return false;
            }

            left++;

```

```

        right--;
    }

    return true;
}
};

```

12.3 Approach 2 — Reverse Half Number

Avoid overflow by reversing only half.

```

class Solution {
public:
    bool isPalindrome(int x) {
        if (x < 0) {
            return false;
        }

        if (x % 10 == 0 && x != 0) {
            return false;
        }

        int reversedHalf = 0;

        while (x > reversedHalf) {
            reversedHalf = reversedHalf * 10 + x % 10;
            x /= 10;
        }

        return x == reversedHalf || x == reversedHalf / 10;
    }
};

```

13. Full Problem Walkthrough 4 — LeetCode 5: Longest Palindromic Substring

13.1 Problem Summary

Given string *s*, return longest palindromic substring.

Example:

```
s = "babad"
```

Output:

```
"bab" or "aba"
```

13.2 Pattern

Expand around every center.

There are:

n odd centers
n even centers

13.3 C++ Code

```
class Solution {
public:
    int bestStart = 0;
    int bestLen = 0;

    void expand(string& s, int left, int right) {
        while (left >= 0 && right < s.size() && s[left] == s[right])
        {
            int len = right - left + 1;

            if (len > bestLen) {
                bestLen = len;
                bestStart = left;
            }

            left--;
            right++;
        }
    }

    string longestPalindrome(string s) {
        for (int i = 0; i < s.size(); i++) {
            expand(s, i, i);
            expand(s, i, i + 1);
        }

        return s.substr(bestStart, bestLen);
    }
};
```

13.4 Complexity

Time:

$O(n^2)$

Space:

$O(1)$

13.5 Why Not Sliding Window?

Palindrome property is not monotonic.

If a window is palindrome, expanding it may or may not remain palindrome.

So sliding window is not natural here.

14. Full Problem Walkthrough 5 —

LeetCode 647: Palindromic Substrings

14.1 Problem Summary

Count all palindromic substrings.

Example:

```
s = "aaa"
```

Palindromic substrings:

```
"a", "a", "a", "aa", "aa", "aaa"
```

Answer:

```
6
```

14.2 Pattern

Expand around every center.

Every successful expansion gives one palindrome.

14.3 C++ Code

```
class Solution {
public:
    int expand(string& s, int left, int right) {
        int count = 0;

        while (left >= 0 && right < s.size() && s[left] == s[right])
        {
            count++;
            left--;
            right++;
        }

        return count;
    }

    int countSubstrings(string s) {
        int answer = 0;

        for (int i = 0; i < s.size(); i++) {
            answer += expand(s, i, i);
            answer += expand(s, i, i + 1);
        }

        return answer;
    }
};
```

14.4 Complexity

Time:

$O(n^2)$

Space:

$O(1)$

15. Full Problem Walkthrough 6 — Longest Palindromic Substring With DP

15.1 When Use DP Instead of Expand?

DP is useful when:

- you also need a palindrome table
- the problem combines palindrome checks with other DP
- you want to reuse palindrome results

15.2 State

`isPal[l][r] = true` if `s[l..r]` is palindrome

15.3 Transition

`s[l] == s[r]` and inner substring is palindrome

For length ≤ 2 , matching ends are enough.

15.4 C++ Code

```
class Solution {
public:
    string longestPalindrome(string s) {
        int n = s.size();

        vector<vector<bool>> isPal(n, vector<bool>(n, false));

        int bestStart = 0;
        int bestLen = 1;

        for (int len = 1; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if (s[l] == s[r]) {
                    if (len <= 2 || isPal[l + 1][r - 1]) {
                        isPal[l][r] = true;

                        if (len > bestLen) {
                            bestLen = len;
                        }
                    }
                }
            }
        }

        return s.substr(bestStart, bestLen);
    }
};
```

```

        bestStart = 1;
    }
}
}
}
return s.substr(bestStart, bestLen);
}
};

```

15.5 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

16. Full Problem Walkthrough 7 — LeetCode 516: Longest Palindromic Subsequence

16.1 Problem Summary

Return length of longest palindromic subsequence.

Example:

`s = "bbbab"`

Answer:

4

because:

"bbbb"

16.2 Pattern

This is interval DP.

16.3 State

`dp[l][r]` = longest palindromic subsequence length in `s[l..r]`

16.4 Transition

If ends match:

`dp[l][r] = 2 + dp[l + 1][r - 1]`

Else:

`dp[l][r] = max(dp[l + 1][r], dp[l][r - 1])`

16.5 C++ Code

```
class Solution {
public:
    int longestPalindromeSubseq(string s) {
        int n = s.size();

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if (s[l] == s[r]) {
                    dp[l][r] = 2 + dp[l + 1][r - 1];
                } else {
                    dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
                }
            }
        }

        return dp[0][n - 1];
    }
};
```

16.6 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

17. Full Problem Walkthrough 8 — LeetCode 1312: Minimum Insertion Steps to Make a String Palindrome

17.1 Problem Summary

Return minimum insertions needed to make s palindrome.

Example:

s = "mbadm"

Answer:

17.2 Key Insight

Keep the longest palindromic subsequence unchanged.

Insert characters around the rest.

Therefore:

minimum insertions = $n - \text{LPS}$

17.3 C++ Code

```
class Solution {
public:
    int minInsertions(string s) {
        int n = s.size();

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if (s[l] == s[r]) {
                    dp[l][r] = 2 + dp[l + 1][r - 1];
                } else {
                    dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
                }
            }
        }

        return n - dp[0][n - 1];
    }
};
```

17.4 Alternative DP

Direct recurrence:

```
if s[l] == s[r]:
    dp[l][r] = dp[l + 1][r - 1]
else:
    dp[l][r] = 1 + min(dp[l + 1][r], dp[l][r - 1])
```

This directly computes minimum insertions.

18. Full Problem Walkthrough 9 —

LeetCode 131: Palindrome Partitioning

18.1 Problem Summary

Return all ways to partition string so every substring is palindrome.

Example:

```
s = "aab"
```

Output:

```
["a","a","b"]  
["aa","b"]
```

18.2 Pattern

Backtracking.

At every index, choose a palindromic prefix.

18.3 Optimization

Precompute palindrome table to avoid checking repeatedly.

18.4 C++ Code

```
class Solution {  
public:  
    vector<vector<string>> result;  
    vector<string> path;  
    vector<vector<bool>> isPal;  
  
    void buildPalindromeTable(string& s) {  
        int n = s.size();  
  
        isPal.assign(n, vector<bool>(n, false));  
  
        for (int len = 1; len <= n; len++) {  
            for (int l = 0; l + len - 1 < n; l++) {  
                int r = l + len - 1;  
  
                if (s[l] == s[r]) {  
                    if (len <= 2 || isPal[l + 1][r - 1]) {  
                        isPal[l][r] = true;  
                    }  
                }  
            }  
        }  
    }  
  
    void backtrack(string& s, int start) {  
        if (start == s.size()) {
```

```

        result.push_back(path);
        return;
    }

    for (int end = start; end < s.size(); end++) {
        if (isPal[start][end]) {
            path.push_back(s.substr(start, end - start + 1));

            backtrack(s, end + 1);

            path.pop_back();
        }
    }
}

vector<vector<string>> partition(string s) {
    buildPalindromeTable(s);

    backtrack(s, 0);

    return result;
}
};

```

18.5 Complexity

There can be exponentially many partitions.

Time:

$O(2^n * n)$

roughly output-dependent.

Space:

$O(n^2)$

for palindrome table.

19. Full Problem Walkthrough 10 — LeetCode 132: Palindrome Partitioning II

19.1 Problem Summary

Return minimum cuts needed so every substring is palindrome.

Example:

`s = "aab"`

Minimum partition:

`aa | b`

Answer:

1

19.2 Pattern

Palindrome table + 1D DP.

19.3 State

$dp[i]$ = minimum cuts needed for $s[0..i]$

19.4 Transition

If $s[0..i]$ is palindrome:

$dp[i] = 0$

Otherwise try last palindrome segment:

$dp[i] = \min(dp[j] + 1)$
where $s[j + 1..i]$ is palindrome

19.5 C++ Code

```
class Solution {
public:
    int minCut(string s) {
        int n = s.size();

        vector<vector<bool>> isPal(n, vector<bool>(n, false));

        for (int len = 1; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if (s[l] == s[r]) {
                    if (len <= 2 || isPal[l + 1][r - 1]) {
                        isPal[l][r] = true;
                    }
                }
            }
        }

        vector<int> dp(n, 0);

        for (int i = 0; i < n; i++) {
            if (isPal[0][i]) {
                dp[i] = 0;
            } else {
                dp[i] = i;

                for (int j = 0; j < i; j++) {
                    if (isPal[j + 1][i]) {
                        dp[i] = min(dp[i], dp[j] + 1);
                    }
                }
            }
        }
    }
};
```

```

        }
    }
    return dp[n - 1];
}
};

```

19.6 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

20. Full Problem Walkthrough 11 — LeetCode 214: Shortest Palindrome

20.1 Problem Summary

Given string s , add characters in front to make it palindrome.

Return shortest possible palindrome.

Example:

$s = \text{"aacecaaa"}$

Answer:

"aaacecaaa"

20.2 Key Insight

We need longest palindromic prefix.

If longest palindromic prefix has length L , then suffix $s[L..]$ must be reversed and added to front.

20.3 KMP Trick

Build:

$s + \text{"\#"} + \text{reverse}(s)$

The last LPS value gives length of longest palindromic prefix.

20.4 C++ Code

```

class Solution {
public:
    vector<int> buildLPS(string p) {
        vector<int> lps(p.size(), 0);

        int len = 0;

        for (int i = 1; i < p.size(); ) {
            if (p[i] == p[len]) {

```

```

        len++;
        lps[i] = len;
        i++;
    } else {
        if (len > 0) {
            len = lps[len - 1];
        } else {
            i++;
        }
    }
}

return lps;
}

string shortestPalindrome(string s) {
    string rev = s;
    reverse(rev.begin(), rev.end());

    string combined = s + "#" + rev;

    vector<int> lps = buildLPS(combined);

    int palPrefixLen = lps.back();

    string suffix = s.substr(palPrefixLen);
    reverse(suffix.begin(), suffix.end());

    return suffix + s;
}
};

```

20.5 Complexity

Time:

$O(n)$

Space:

$O(n)$

21. Full Problem Walkthrough 12 — LeetCode 1328: Break a Palindrome

21.1 Problem Summary

Given a palindromic string, replace exactly one character to make it not palindrome and lexicographically smallest.

If impossible, return empty string.

Example:

```
"abccba"
```

Change first non-a in first half to a:

```
"aaccba"
```

21.2 Greedy Insight

To make lexicographically smallest string:

- Change first non-a character in first half to a.
- If all first half characters are a, change last character to b.

21.3 C++ Code

```
class Solution {
public:
    string breakPalindrome(string palindrome) {
        int n = palindrome.size();

        if (n == 1) {
            return "";
        }

        for (int i = 0; i < n / 2; i++) {
            if (palindrome[i] != 'a') {
                palindrome[i] = 'a';
                return palindrome;
            }
        }

        palindrome[n - 1] = 'b';

        return palindrome;
    }
};
```

22. Full Problem Walkthrough 13 — LeetCode 1216: Valid Palindrome III

22.1 Problem Summary

Given string *s* and integer *k*, return true if *s* can become palindrome after removing at most *k* characters.

22.2 Key Insight

Minimum deletions needed:

$n - \text{longest palindromic subsequence}$

So valid if:

$n - \text{LPS} \leq k$

22.3 C++ Code

```

class Solution {
public:
    bool isValidPalindrome(string s, int k) {
        int n = s.size();

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if (s[l] == s[r]) {
                    dp[l][r] = 2 + dp[l + 1][r - 1];
                } else {
                    dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
                }
            }
        }

        return n - dp[0][n - 1] <= k;
    }
};

```

23. Full Problem Walkthrough 14 — Count Different Palindromic Subsequences Preview

23.1 Why This Is Hard

Counting palindromic substrings is easy with center expansion.

Counting distinct palindromic subsequences is much harder because duplicates cause overcounting.

Example:

"aaa"

Palindromic subsequences by positions are many, but distinct values are:

"a"

"aa"

"aaa"

23.2 State

`dp[l][r]` = number of distinct palindromic subsequences in `s[l..r]`

23.3 Challenge

If `s[l] == s[r]`, you must know whether the same character appears inside the interval. This changes how much you add/subtract.

23.4 Lesson

This belongs to advanced interval DP and should be revisited after more practice.

24. Full Problem Walkthrough 15 — Palindrome Pairs Preview

24.1 Problem Summary

Given list of words, find pairs `i, j` such that:

`words[i] + words[j]`

is palindrome.

24.2 Key Idea

For each word, split it into:

prefix + suffix

If prefix is palindrome, need reversed suffix as another word.

If suffix is palindrome, need reversed prefix as another word.

24.3 High-Level Code Idea

```
for each word:
    for every split:
        prefix = word[0..split-1]
        suffix = word[split..end]

        if prefix is palindrome:
            look for reverse(suffix)

        if suffix is palindrome:
            look for reverse(prefix)
```

24.4 Techniques

Can be solved with:

- hashmap
- trie
- rolling hash

This is a hard string problem.

25. Full Problem Walkthrough 16 — Manacher's Algorithm Preview

25.1 What Is Manacher's Algorithm?

Manacher's Algorithm finds all palindromic radii in linear time.
It can solve longest palindromic substring in:

$O(n)$

while expand-around-center is:

$O(n^2)$

25.2 Should You Learn It?

For normal interviews:

expand around center is usually enough

For competitive programming or very hard string rounds:

Manacher is useful

25.3 Why Not Cover Fully Here?

It is implementation-heavy and less commonly expected than KMP/DP/center expansion.

But you should know it exists.

26. Common Palindrome Edge Cases

26.1 Empty String

Usually palindrome.

But some problems do not include empty input.

26.2 One Character

Always palindrome.

Special case in Break a Palindrome:

```
return ""
```

26.3 Even vs Odd Length

Always consider both centers.

26.4 All Same Characters

Example:

```
"aaaa"
```

Many palindromic substrings.

26.5 Case and Non-Alphanumeric

Valid Palindrome ignores them.

Other problems usually do not.

26.6 Deletion Allowed

If one deletion is allowed, do not greedily skip only one side without checking both.

27. Common Palindrome Mistakes

Mistake 1 — Confusing Substring and Subsequence

Longest Palindromic Substring and Longest Palindromic Subsequence are different problems.

Mistake 2 — Forgetting Even Centers

For "abba", center is between two b's.

Mistake 3 — Wrong DP Fill Order

For $dp[l][r]$, fill by length.

Mistake 4 — Rechecking Palindromes in Backtracking

Precompute table for Palindrome Partitioning.

Mistake 5 — Shortest Palindrome Wrong Prefix

Need longest palindromic prefix, not longest palindromic substring.

Mistake 6 — Break Palindrome Changing Second Half First

To minimize lexicographically, change earliest possible character.

28. Interview Explanation Templates

28.1 Basic Palindrome

I use two pointers from both ends and compare characters while moving inward.

28.2 Valid Palindrome II

At the first mismatch, since only one deletion is allowed, the only possible fixes are skipping the left character or skipping the right character.

28.3 Longest Palindromic Substring

Every palindrome expands around a center. I try every odd and even center and keep the longest expansion.

28.4 Palindromic Substrings

Every successful expansion around a center represents one palindromic substring, so I count all expansions.

28.5 Longest Palindromic Subsequence

I use interval DP. If the two ends match, they can be part of the subsequence. Otherwise, I skip one end and take the better result.

28.6 Palindrome Partitioning II

I precompute which substrings are palindromes, then use prefix DP where $dp[i]$ is the minimum cuts for $s[0..i]$.

28.7 Shortest Palindrome

I need the longest palindromic prefix. I find it using KMP on $s + \text{separator} + \text{reverse}(s)$, then add the reverse of the remaining suffix to the front.

29. Complexity Patterns

Problem	Time	Space
Basic palindrome check	$O(n)$	$O(1)$
Valid Palindrome II	$O(n)$	$O(1)$

Problem	Time	Space
Palindrome Number	$O(\log n)$	$O(1)$
Longest Palindromic Substring, center	$O(n^2)$	$O(1)$
Longest Palindromic Substring, DP	$O(n^2)$	$O(n^2)$
Count Palindromic Substrings	$O(n^2)$	$O(1)$
Longest Palindromic Subsequence	$O(n^2)$	$O(n^2)$
Minimum Insertions	$O(n^2)$	$O(n^2)$
Palindrome Partitioning	output exponential	$O(n^2)$
Palindrome Partitioning II	$O(n^2)$	$O(n^2)$
Shortest Palindrome KMP	$O(n)$	$O(n)$
Palindrome Pairs	$O(\text{words} * \text{length}^2)$ typical	$O(\text{words} * \text{length})$
Manacher	$O(n)$	$O(n)$

30. Expanded Practice Set

30.1 Must-Do Easy / Medium

- 28. Valid Palindrome
- 29. Valid Palindrome II
- 30. Palindrome Number
- 31. Longest Palindromic Substring
- 32. Palindromic Substrings
- 33. Longest Palindromic Subsequence
- 34. Minimum Insertion Steps to Make a String Palindrome
- 35. Palindrome Partitioning
- 36. Palindrome Partitioning II
- 37. Break a Palindrome

30.2 Strong Interview Practice

- 38. Shortest Palindrome
- 39. Valid Palindrome III
- 40. Palindrome Pairs
- 41. Count Different Palindromic Subsequences
- 42. Longest Chunked Palindrome Decomposition
- 43. Maximum Product of the Length of Two Palindromic Subsequences
- 44. Split Two Strings to Make Palindrome

30.3 Hard / Stretch

- 45. Manacher's Algorithm
 - 46. Palindrome Pairs with Trie
 - 47. Count Different Palindromic Subsequences
 - 48. Palindrome Partitioning III
 - 49. Super Palindromes
 - 50. Largest Palindrome Product
-

31. Suggested Study Order

Step 1 — Basic Checks

Solve:

- 51. Valid Palindrome
- 52. Valid Palindrome II
- 53. Palindrome Number

Goal:

Master two pointers.

Step 2 — Center Expansion

Solve:

- 54. Longest Palindromic Substring
- 55. Palindromic Substrings

Goal:

Master odd/even centers.

Step 3 — Interval DP

Solve:

- 56. Longest Palindromic Subsequence
- 57. Minimum Insertions
- 58. Valid Palindrome III

Goal:

Master subsequence palindrome DP.

Step 4 — Partitioning

Solve:

- 59. Palindrome Partitioning
- 60. Palindrome Partitioning II

Goal:

Combine palindrome table with backtracking/DP.

Step 5 — Prefix Palindrome

Solve:

- 61. Shortest Palindrome

Goal:

Use KMP for longest palindromic prefix.

Step 6 — Advanced

Study:

- 62. Palindrome Pairs
- 63. Count Different Palindromic Subsequences
- 64. Manacher

Goal:

Recognize hard palindrome patterns.

32. Pattern Decision Table

Problem Clue	Technique
entire string palindrome	two pointers
ignore punctuation/case	filtered two pointers
delete at most one char	two pointers + helper
longest contiguous palindrome	expand center
count contiguous palindromes	expand center
repeated palindrome checks	palindrome DP table
longest palindrome after deletions	LPS DP
min insertions/deletions to palindrome	n - LPS
all palindrome partitions	backtracking + isPal
minimum palindrome cuts	isPal + prefix DP
add front chars to palindrome	KMP longest pal prefix
palindrome word pairs	hashmap/trie
linear longest palindrome substring	Manacher

33. Mini Problem 1 — Split Two Strings to Make Palindrome

Given a and b, check if splitting at some index can make:

a_prefix + b_suffix

or:

b_prefix + a_suffix

a palindrome.

Key helper:

```
bool check(string& a, string& b) {
    int left = 0;
    int right = a.size() - 1;

    while (left < right && a[left] == b[right]) {
        left++;
        right--;
    }
```

```
    }  
  
    return isPal(a, left, right) || isPal(b, left, right);  
}
```

This is advanced two-pointer palindrome logic.

34. Mini Problem 2 — Longest Palindrome Length From Characters

Problem:

Given string *s*, return length of longest palindrome that can be built using its letters.

Idea:

Use all even counts.

For odd counts, use count - 1 and allow one center.

```
class Solution {  
public:  
    int longestPalindrome(string s) {  
        vector<int> count(128, 0);  
  
        for (char c : s) {  
            count[c]++;  
        }  
  
        int length = 0;  
        bool hasOdd = false;  
  
        for (int x : count) {  
            length += (x / 2) * 2;  
  
            if (x % 2 == 1) {  
                hasOdd = true;  
            }  
        }  
  
        if (hasOdd) {  
            length++;  
        }  
  
        return length;  
    }  
};
```

35. Mini Problem 3 — Can Permute Palindrome

A string can be permuted into a palindrome if at most one character has odd frequency.

```
bool canPermutePalindrome(string s) {
    vector<int> count(128, 0);

    for (char c : s) {
        count[c]++;
    }

    int odd = 0;

    for (int x : count) {
        if (x % 2 == 1) {
            odd++;
        }
    }

    return odd <= 1;
}
```

36. Mini Problem 4 — Palindrome Linked List Preview

Although not string, it uses palindrome logic.

Approach:

65. Find middle with slow/fast pointers.

66. Reverse second half.

67. Compare both halves.

This combines linked list + palindrome.

37. Common Edge Case Checklist

Before submitting palindrome code, test:

```
" "  
"a"  
"aa"  
"ab"  
"aba"  
"abba"  
"abcba"  
"abca"  
"aaaa"  
"A man, a plan, a canal: Panama"  
"race a car"  
numbers ending in 0
```

single-character palindrome for break problem
all 'a' palindrome for break problem

38. Debugging Palindrome Problems

If your palindrome solution fails, check:

- 68. Did you handle even-length palindromes?
- 69. Did you distinguish substring vs subsequence?
- 70. Is your two-pointer skip logic correct?
- 71. Are non-alphanumeric characters ignored if required?
- 72. Did you lowercase characters if required?
- 73. Is DP filled by length?
- 74. Are base cases for length 1 and 2 correct?
- 75. Are you recomputing palindrome checks unnecessarily?
- 76. For shortest palindrome, are you finding prefix palindrome, not any palindrome?
- 77. For partitioning, are substr indices correct?

39. What to Master Before Chapter 46

Before moving to Trie, make sure you can:

- Check palindrome with two pointers
- Solve Valid Palindrome
- Solve Valid Palindrome II
- Solve Longest Palindromic Substring
- Solve Palindromic Substrings
- Explain odd/even center expansion
- Solve Longest Palindromic Subsequence
- Use palindrome DP table
- Solve Palindrome Partitioning
- Solve Palindrome Partitioning II
- Solve Shortest Palindrome with KMP
- Recognize when advanced tools like trie/rolling hash/Manacher may help

40. Final Cheat Sheet

Basic Check

```
while (left < right) {  
    if (s[left] != s[right]) {  
        return false;  
    }  
  
    left++;  
    right--;  
}
```

Delete At Most One

```
if (s[left] != s[right]) {  
    return check(left + 1, right) || check(left, right - 1);  
}
```

Expand Center

```
while (left >= 0 && right < n && s[left] == s[right]) {  
    left--;  
    right++;  
}
```

Count Palindromic Substrings

```
answer += expand(i, i);  
answer += expand(i, i + 1);
```

Palindrome Table

```
isPal[l][r] = s[l] == s[r] && (len <= 2 || isPal[l + 1][r - 1]);
```

LPS

```
if (s[l] == s[r]) {  
    dp[l][r] = 2 + dp[l + 1][r - 1];  
} else {  
    dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);  
}
```

Minimum Insertions

```
answer = n - LPS;
```

Minimum Cuts

```
if (isPal[0][i]) {  
    dp[i] = 0;  
} else if (isPal[j + 1][i]) {  
    dp[i] = min(dp[i], dp[j] + 1);  
}
```

Shortest Palindrome

```
combined = s + "#" + reverse(s)  
longest palindromic prefix = lps.back()
```

41. Core Takeaway

Palindrome problems are symmetry problems.

The first decision is:

substring or subsequence?

If substring:

- two pointers for checking
- expand around center for longest/count

- DP table for repeated checks

If subsequence:

- interval DP

If partitioning:

- palindrome table + backtracking or prefix DP

If prefix palindrome:

- KMP or rolling hash

Most palindrome mistakes come from:

- forgetting even-length centers
- confusing substring with subsequence
- wrong DP fill order
- not handling the first mismatch carefully

Once you classify the palindrome type, the solution pattern becomes much clearer.